Antes de dibujar, pregunta

Una relación describe una regla del negocio. El JOIN utiliza esa relación para construir una vista de los datos.

1. ¿Qué representa exactamente una fila?

Estudiante: una persona. Contacto: una dirección de correo de esa persona. Calificación: la nota de una persona en un examen. Esta unidad se llama granularidad.

2. Pregunta en las dos direcciones

Regla	Cardinalidad	Dónde va la FK
Un estudiante tiene como máximo un contacto único	1 : 0..1	En contacto_unico; además PK o UNIQUE
Un estudiante puede tener varios correos	1 : 0..N	En contactos; la FK se puede repetir
Un estudiante hace varios exámenes; un examen tiene varios estudiantes	N : M	En calificaciones: dos FK

Mínimo y máximo son distintos: 0..N permite ninguno; 1..N exige al menos uno. Una FK obliga al hijo a apuntar a un padre válido; no obliga al padre a tener hijos.

3. Identifica, conecta y restringe

PK: identifica una fila, es única y no nula. FK: referencia una clave de otra tabla. UNIQUE: impide repetir valores o combinaciones. NOT NULL: obliga a informar el dato.

¿Qué ganamos al relacionar?

Guardamos cada hecho en su lugar: cambiar el nombre de Ana una vez actualiza su identidad para todos sus contactos y notas. Las FK impiden referencias inexistentes; las claves evitan duplicados. Podemos crecer sin añadir correo_4, correo_5…

Coste: más tablas y consultas con JOIN; hay que elegir índices y reglas de borrado. Separar por separar no mejora el diseño: un único email obligatorio puede vivir en estudiantes.

02 · 1 A 1

Una extensión del estudiante

Compartir la clave es una forma sencilla de impedir que haya dos contactos únicos para la misma persona.




```
CREATE SCHEMA tutorial_relaciones;
SET search_path TO tutorial_relaciones;

CREATE TABLE estudiantes (
  id integer PRIMARY KEY,
  nombre text NOT NULL
);
CREATE TABLE contacto_unico (
  estudiante_id integer PRIMARY KEY
    REFERENCES estudiantes(id),
  email text NOT NULL
);
INSERT INTO estudiantes VALUES (1,'Ana'),(2,'Bruno'),(3,'Carla');
INSERT INTO contacto_unico VALUES
  (1,'ana@uni.test'),(2,'bruno@uni.test');
```

estudiantes	contacto_unico
1 · Ana	1 · ana@uni.test
2 · Bruno	2 · bruno@uni.test
3 · Carla	Sin fila de contacto

La FK también es la PK de la tabla hija. Sus valores referencian la PK de estudiantes. Otra posibilidad: una PK propia y estudiante_id UNIQUE NOT NULL.

La restricción implementa uno a cero o uno: Carla puede existir sin contacto. No garantiza un contacto obligatorio para todos. Separar tiene sentido por permisos, atributos opcionales o un ciclo de vida distinto.

Juntar sin multiplicar

Con una coincidencia como máximo, la tabla hija se comporta como una extensión de columnas.

```
SELECT e.id, e.nombre, u.email
FROM estudiantes AS e
JOIN contacto_unico AS u ON u.estudiante_id = e.id
ORDER BY e.id;
```

id	nombre	email
1	Ana	ana@uni.test
2	Bruno	bruno@uni.test

INNER JOIN (o simplemente JOIN) conserva coincidencias. Carla no aparece porque no tiene contacto. La relación 1:1 evita multiplicar filas, pero no evita perder las que no coinciden.

LEFT JOIN: todos los estudiantes

```
SELECT e.id, e.nombre, u.email
FROM estudiantes AS e
LEFT JOIN contacto_unico AS u ON u.estudiante_id = e.id
ORDER BY e.id;
```

id	nombre	email
1	Ana	ana@uni.test
2	Bruno	bruno@uni.test
3	Carla	NULL

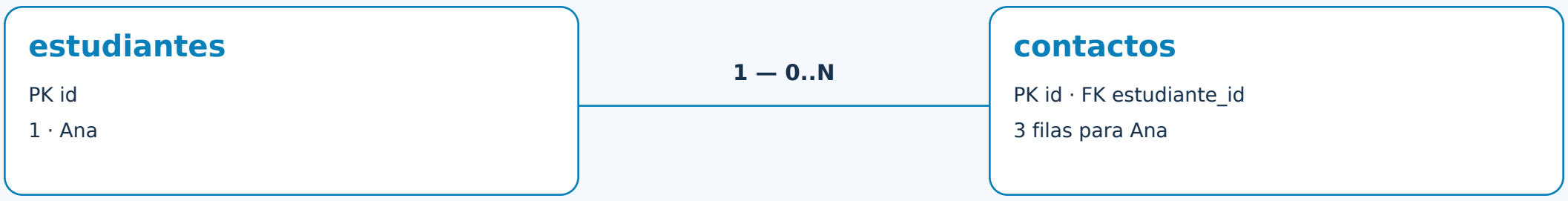
NULL significa aquí que el JOIN no encontró fila hija; no es un texto ni una cadena vacía. Compruébalo con IS NULL, nunca con = NULL.

No necesitas un JOIN para cada dato

Si email fuese un atributo simple de estudiantes, SELECT id, nombre, email bastaría. La separación 1:1 es una decisión de diseño, no una obligación de normalización.

La FK ahora puede repetirse

Cada correo tiene identidad propia; varios correos pueden pertenecer al mismo estudiante.




```
CREATE TABLE contactos (
  id integer PRIMARY KEY,
  estudiante_id integer NOT NULL REFERENCES estudiantes(id),
  email text NOT NULL,
  prioridad integer NOT NULL CHECK (prioridad > 0),
  principal boolean NOT NULL DEFAULT false,
  UNIQUE (estudiante_id, email)
);
CREATE UNIQUE INDEX un_principal_por_estudiante
  ON contactos(estudiante_id) WHERE principal;

INSERT INTO contactos VALUES
 (11,1,'ana@uni.test',1,true),
 (12,1,'ana@casa.test',2,false),
 (13,1,'ana@trabajo.test',3,false),
 (21,2,'bruno@uni.test',1,true);
```

```
SELECT e.id, e.nombre, c.email
FROM estudiantes e
JOIN contactos c ON c.estudiante_id = e.id
WHERE e.id = 1
ORDER BY c.prioridad, c.id;
```

id	nombre	email
1	Ana	ana@uni.test
1	Ana	ana@casa.test
1	Ana	ana@trabajo.test

Tres filas correctas: la unidad del resultado es ahora estudiante-contacto. Sirve para listar, auditar o procesar cada correo. El nombre se repite; las filas completas no son duplicados.

El índice parcial permite como máximo un principal, no exige que exista uno. La prioridad admite empates; id servirá para desempatar.

05 · ELEGIR UNO

Una fila por estudiante

Antes de reducir filas, decide qué dato quieres conservar. Elegir implica descartar los demás contactos del resultado.

A · Solo el correo marcado como principal

```
SELECT e.id, e.nombre, c.email
FROM estudiantes e
LEFT JOIN contactos c
  ON c.estudiante_id = e.id AND c.principal
ORDER BY e.id;
```

Ana → ana@uni.test; Bruno → bruno@uni.test; Carla → NULL. Si hay contactos pero ninguno principal, también sale NULL. La garantía de no multiplicar la aporta el índice UNIQUE parcial.

B · El primero por prioridad, con LIMIT 1 por persona

```
SELECT e.id, e.nombre, elegido.email
FROM estudiantes e
LEFT JOIN LATERAL (
  SELECT c.email
  FROM contactos c
  WHERE c.estudiante_id = e.id
  ORDER BY c.prioridad, c.id
  LIMIT 1
) AS elegido ON true
ORDER BY e.id;
```

LATERAL permite que la subconsulta use e.id. Se elige un contacto por estudiante; si no hay ninguno, LEFT JOIN conserva al estudiante con NULL.

C · LIMIT 1 global: otro objetivo

```
SELECT e.id, e.nombre, c.email
FROM estudiantes e JOIN contactos c ON c.estudiante_id = e.id
ORDER BY e.id, c.prioridad, c.id
LIMIT 1;
```

Devuelve una sola fila en total: Ana y ana@uni.test. No una por estudiante. Sin ORDER BY, el elegido no queda determinado.

Ranking, agrupación y existencia

Tres necesidades distintas: escoger uno, resumir todos o comprobar si existe alguno.

D · Ranking por estudiante: ROW_NUMBER

```
WITH ordenados AS (
  SELECT c.*, ROW_NUMBER() OVER (
    PARTITION BY estudiante_id ORDER BY prioridad, id
  ) AS rn
  FROM contactos c
)
SELECT e.id, e.nombre, o.email
FROM estudiantes e
LEFT JOIN ordenados o ON o.estudiante_id = e.id AND o.rn = 1
ORDER BY e.id;
```

PARTITION BY reinicia la numeración para cada persona. rn = 1 elige el primero; el resultado coincide con la estrategia B.

E · Todos los correos en una sola fila de salida

```
SELECT e.id, e.nombre,
       COUNT(c.id) AS total,
       COALESCE(ARRAY_AGG(c.email ORDER BY c.prioridad, c.id)
         FILTER (WHERE c.id IS NOT NULL), ARRAY[]::text[]) AS emails
FROM estudiantes e
LEFT JOIN contactos c ON c.estudiante_id = e.id
GROUP BY e.id, e.nombre
ORDER BY e.id;
```

Ana: 3 y sus tres correos; Bruno: 1; Carla: 0 y array vacío. Agregar para presentar no exige guardar arrays en las tablas de origen.

F · Solo quiero estudiantes con algún contacto

```
SELECT e.id, e.nombre
FROM estudiantes e
WHERE EXISTS (
  SELECT 1 FROM contactos c WHERE c.estudiante_id = e.id
)
ORDER BY e.id;
```

Devuelve Ana y Bruno una sola vez. DISTINCT también puede quitar filas idénticas proyectadas, pero no decide qué email es el correcto.

¿Un correo por fila o una lista?

La atomización depende del dominio y de cómo necesitas consultar y validar sus valores.

En el enfoque clásico de primera forma normal (1NF), una lista de correos dentro de una celda es un grupo repetido: si cada correo es un hecho independiente, conviene una fila por correo. PostgreSQL también admite arrays como valores tipados; la discusión de atomicidad tiene ese matiz.

```
CREATE TABLE contactos_array (
  estudiante_id integer PRIMARY KEY REFERENCES estudiantes(id),
  emails text[] NOT NULL DEFAULT ARRAY[]::text[]
);
INSERT INTO contactos_array VALUES
 (1,ARRAY['ana@uni.test','ana@casa.test','ana@trabajo.test']),
 (2,ARRAY['bruno@uni.test']), (3,ARRAY[]::text[]);

-- Primer elemento: convención de prioridad por posición.
SELECT estudiante_id, emails[1] AS preferido
FROM contactos_array ORDER BY estudiante_id;

-- Buscar dentro del array, sin salir de SQL.
SELECT estudiante_id FROM contactos_array
WHERE 'ana@casa.test' = ANY(emails);

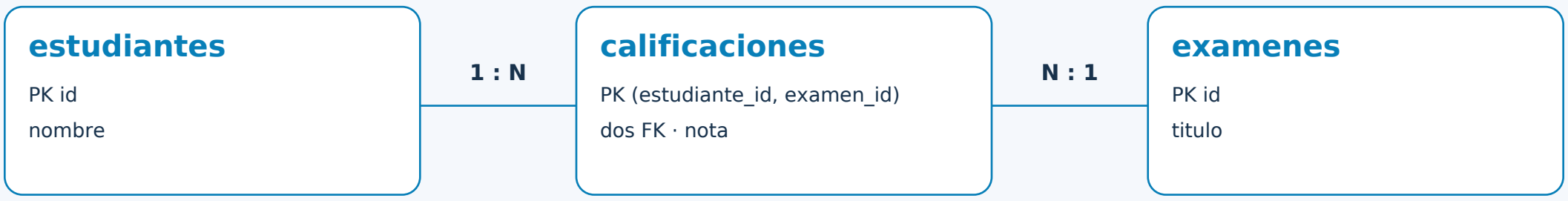
-- Convertir la lista en filas y conservar el orden.
SELECT a.estudiante_id, u.email, u.posicion
FROM contactos_array a
CROSS JOIN LATERAL unnest(a.emails) WITH ORDINALITY
  AS u(email, posicion)
ORDER BY a.estudiante_id, u.posicion;
```

Una fila por correo	Un array por estudiante
Fácil añadir prioridad, verificación y fecha por correo.	Cómodo si siempre lees y sustituyes la lista completa.
UNIQUE por estudiante/email y FK hacia cada contacto.	Unicidad interna y referencias por elemento requieren más trabajo.
Orden explícito mediante prioridad y ORDER BY.	El orden existe, pero mover posiciones puede cambiar el principal.
Más filas y posibles JOIN.	Acceso por índice, ANY y unnest dentro de SQL; no requiere código externo.

Una lista vacía genera cero filas con CROSS JOIN unnest. Para conservarla, usa LEFT JOIN LATERAL unnest(...) AS u(email) ON true. No confundas array vacío con NULL.

Las calificaciones son el vínculo

Suponemos una sola nota por estudiante y examen. Si hay reintentos, habría que modelarlos también.




```
CREATE TABLE examenes (
  id integer PRIMARY KEY,
  titulo text NOT NULL
);
CREATE TABLE calificaciones (
  estudiante_id integer REFERENCES estudiantes(id),
  examen_id integer REFERENCES examenes(id),
  nota numeric(4,2) NOT NULL CHECK (nota BETWEEN 0 AND 10),
  PRIMARY KEY (estudiante_id, examen_id)
);
INSERT INTO examenes VALUES (101,'SQL'),(102,'Modelado');
INSERT INTO calificaciones VALUES (1,101,8),(1,102,9),(2,101,7);

SELECT e.nombre, x.titulo, g.nota
FROM calificaciones g
JOIN estudiantes e ON e.id = g.estudiante_id
JOIN examenes x ON x.id = g.examen_id
ORDER BY e.id, x.id;
```

nombre	examen	nota
Ana	SQL	8
Ana	Modelado	9
Bruno	SQL	7

Entre estudiantes y calificaciones hay 1:N; entre exámenes y calificaciones también. Juntas resuelven el N:M entre estudiantes y exámenes. La nota pertenece a la pareja, no solo a la persona ni solo al examen.

La PK compuesta evita dos notas para la misma pareja y hace ambas columnas NOT NULL. Una PK artificial sería posible, pero seguiríamos necesitando UNIQUE(estudiante_id, examen_id).

Que nadie desaparezca

LEFT conserva todas las filas de la izquierda; RIGHT hace lo mismo con las de la derecha.

```
SELECT e.id, e.nombre, g.examen_id, g.nota
FROM estudiantes e
LEFT JOIN calificaciones g ON g.estudiante_id = e.id
ORDER BY e.id, g.examen_id;

-- Misma conservación, con las tablas invertidas:
SELECT e.id, e.nombre, g.examen_id, g.nota
FROM calificaciones g
RIGHT JOIN estudiantes e ON e.id = g.estudiante_id
ORDER BY e.id, g.examen_id;
```

Ambas: Ana/101/8, Ana/102/9, Bruno/101/7, Carla/NULL/NULL. LEFT y RIGHT no eliminan la multiplicidad: Ana conserva dos filas.

Alarma: faltan contactos o calificaciones

```
SELECT e.id, e.nombre,
  NOT EXISTS (SELECT 1 FROM contactos c
              WHERE c.estudiante_id = e.id) AS falta_contacto,
  NOT EXISTS (SELECT 1 FROM calificaciones g
              WHERE g.estudiante_id = e.id) AS sin_nota
FROM estudiantes e ORDER BY e.id;
```

Carla da true / true; Ana y Bruno false / false. No tener nota registrada no demuestra que alguien no hizo el examen: podría estar pendiente de corrección.

Ausencia en un examen concreto: filtra dentro del ON

```
SELECT e.id, e.nombre
FROM estudiantes e
LEFT JOIN calificaciones g
  ON g.estudiante_id = e.id AND g.examen_id = 102
WHERE g.estudiante_id IS NULL
ORDER BY e.id;
```

Devuelve Bruno y Carla. Asumimos que todos debían hacer el examen 102; en un sistema real, partiríamos de las matrículas o convocatorias esperadas.

Trampa: WHERE g.examen_id = 102 después del LEFT JOIN elimina las filas sin coincidencia. FULL OUTER JOIN conserva ambos lados; con una FK válida y obligatoria, no habrá hijos huérfanos en este enlace.

Dos relaciones 1:N a la vez

JOIN crea combinaciones compatibles. No sabe que tú querías contar correos y notas por separado.

```
SELECT e.nombre, c.email, g.examen_id, g.nota
FROM estudiantes e
JOIN contactos c ON c.estudiante_id = e.id
JOIN calificaciones g ON g.estudiante_id = e.id
WHERE e.id = 1
ORDER BY c.id, g.examen_id;
```

correo de Ana	examen	nota
Cada uno de sus 3 correos	101 y 102	8 y 9

La tabla de arriba resume seis filas reales: cada correo se combina con cada nota. COUNT(*) daría 6; SUM(nota), 51 en vez de 17. No hay una relación correo-examen que justifique esas combinaciones para ese informe.

Solución: resumir cada rama antes del JOIN

```
WITH resumen_contactos AS (
  SELECT estudiante_id, COUNT(*) AS n_contactos
  FROM contactos GROUP BY estudiante_id
), resumen_notas AS (
  SELECT estudiante_id, COUNT(*) AS n_notas, AVG(nota) AS media
  FROM calificaciones GROUP BY estudiante_id
)
SELECT e.id, e.nombre,
       COALESCE(c.n_contactos, 0) AS n_contactos,
       COALESCE(g.n_notas, 0) AS n_notas, g.media
FROM estudiantes e
LEFT JOIN resumen_contactos c ON c.estudiante_id = e.id
LEFT JOIN resumen_notas g ON g.estudiante_id = e.id
ORDER BY e.id;
```

estudiante	contactos	notas	media
Ana	3	2	8,5
Bruno	1	1	7
Carla	0	0	NULL

Cada resumen tiene como máximo una fila por estudiante: ahora el JOIN vuelve a ser una extensión. No conviertas una media ausente en 0; sería una nota inventada.

¿De qué clave depende cada dato?

La 2NF trata dependencias parciales respecto de claves candidatas compuestas, no el simple hecho de ver valores repetidos.

Primero 1NF; después, toda la clave

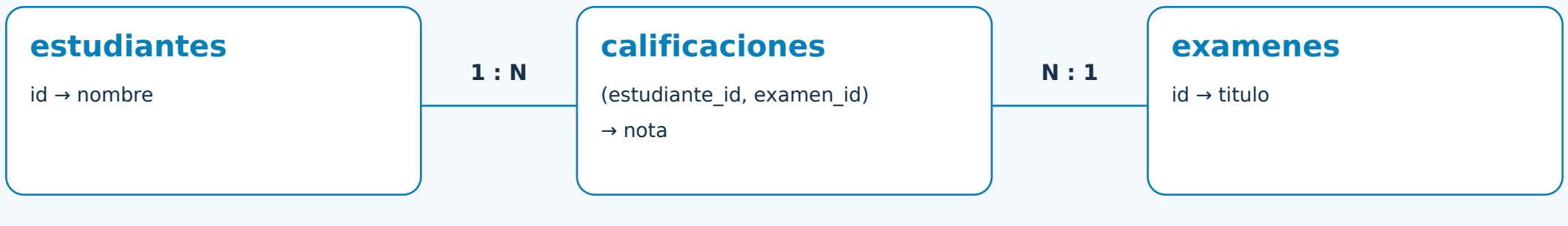
Una relación está en 2NF si está en 1NF y ningún atributo no primo depende de una parte propia de una clave candidata. Un atributo primo pertenece a alguna clave candidata. Hay que revisar todas las claves candidatas, no solo la PK elegida.

Si todas las claves candidatas tienen una sola columna, no puede haber dependencias parciales: cumplida 1NF, se cumple 2NF. Elegir una PK artificial de una columna no borra otras claves candidatas compuestas.

Ejemplo didáctico: almacenar el resultado unido

estudiante_id	examen_id	nombre	titulo	nota
1	101	Ana	SQL	8
1	102	Ana	Modelado	9
2	101	Bruno	SQL	7

Si guardamos esta tabla y su clave candidata es (estudiante_id, examen_id): nombre depende solo de estudiante_id; titulo solo de examen_id; nota depende de la pareja completa. Las dos primeras son dependencias parciales.



Por qué separar; por qué volver a unir

Al almacenar: evitamos renombrar Ana en muchas filas, crear versiones contradictorias de su nombre o perder un examen al borrar su última nota. Cada hecho se actualiza donde corresponde.

Al consultar: reconstruimos un resultado cómodo con JOIN. Puede presentar redundancia sin que el esquema base deje de estar normalizado. No toda unión viola 2NF: depende de sus atributos, claves y dependencias.

La 2NF no es un enemigo del JOIN. Normalizar protege las escrituras; consultar adapta la forma a la lectura. Desnormalizar puede acelerar lecturas, pero exige mantener copias coherentes y medir el beneficio.

Filas, columnas y nombres claros

Relacionar, conservar, seleccionar y proyectar son decisiones distintas.

```
SELECT e.id AS estudiante_id, e.nombre, g.nota
FROM estudiantes AS e
LEFT JOIN calificaciones AS g ON g.estudiante_id = e.id
WHERE e.nombre = 'Ana'
ORDER BY g.examen_id;
```

Paso lógico simplificado	Pregunta que resuelve
FROM / JOIN / ON	¿Qué tablas y qué parejas de filas se conectan?
WHERE	¿Qué filas del resultado conservo?
GROUP BY / agregados	¿A qué nivel resumo?
SELECT (proyección)	¿Qué columnas o expresiones muestro?
ORDER BY / LIMIT	¿En qué orden y cuántas filas devuelvo?

La proyección selecciona columnas; el filtro WHERE selecciona filas. SELECT no elimina repeticiones automáticamente. En este ejemplo Ana aparece con 8 y 9, aunque no mostremos examen_id.

¿tabla.columna o solo columna?

En estudiantes JOIN contactos, ambos tienen id: usa e.id o c.id. Si una columna solo existe en una tabla del ámbito, puedes omitir el prefijo: nombre es válido aquí. SQL no adivina cuando el nombre es ambiguo.

Al declarar estudiantes AS e, usa e.nombre para calificar esa referencia en PostgreSQL. Los alias cortos y consistentes evitan ambigüedades y facilitan añadir tablas. Evita SELECT * en informes estables: su forma puede cambiar al añadir columnas.

Antes de ejecutar, predice el resultado

¿Una fila por estudiante, por contacto o por examen? ¿Quién debe seguir apareciendo si falta información? ¿Elijo un hijo o resumo todos? ¿La unicidad está garantizada por una restricción? ¿Hay dos ramas 1:N que puedan multiplicarse?

Uso del archivo SQL: ejecutar en una base de pruebas PostgreSQL. Crea un esquema nuevo llamado tutorial_relaciones; si ya existe, no lo borra ni lo reemplaza. Los ejemplos de contacto único, múltiple y array coexisten para comparar alternativas; no son tres copias a mantener en producción.

Referencias técnicas: documentación oficial de PostgreSQL, apartados Arrays, Table Expressions y Constraints.
https://www.postgresql.org/docs/current/arrays.html
https://www.postgresql.org/docs/current/queries-table-expressions.html
https://www.postgresql.org/docs/current/ddl-constraints.html